Modern Deep Learning:

Classify Fashion-MNIST with a simple CNN in Keras

By Margaret Maynard-Reid, 4/24/2018

Notebook Overview

Read more about the Fashion-MINST dataset in this paper here (**Fashion-MNIST: a Novel Image Dataset for Benchmarking Machine Learning Algorithms**) The fashion_mnist data: 60,000 train and 10,000 test data with 10 categories. Each gray-scale image is 28x28.

Label Description
0 T-shirt/top
1 Trouser
2 Pullover
3 Dress

4 Coat
5 Sandal
6 Shirt
7 Sneaker
8 Bag
9 Ankle boot

Each gray-scale image is 28x28

Download the fashion_mnist data

First let's install TensorFlow version 1.8.0 and import Tensorflow. Then we download fashion-mnist which is one of the Keras datasets.

In [45]:
```
import tensorflow as tf

import numpy as np
import matplotlib.pyplot as plt

# Load the fashion-mnist pre-shuffled train data and test data
(x_train, y_train), (x_test, y_test) = tf.keras.datasets.fashion_mnist.load_data

# Print the shape of the training data
print("x_train shape:", x_train.shape, "y_train shape:", y_train.shape)

# If code doesn't work, delete this code blcok and use the next code block to lo
```

```
x_train shape: (60000, 28, 28) y_train shape: (60000,)
```

In [46]:
```
# delete this code block if you don't need to load local data files
import tensorflow as tf
import numpy as np
import matplotlib.pyplot as plt
import os
import numpy as np
import gzip
def load_data_fromlocalpath(input_path):
  """Loads the Fashion-MNIST dataset.
  Modified by Henry Huang in 2020/12/24.
  We assume that the input_path should in a correct path address format.
  We also assume that potential users put all the four files in the path.

  Load local data from path ‘input_path’.

  Returns:
      Tuple of Numpy arrays: `(x_train, y_train), (x_test, y_test)`.
  """
  files = [
      'train-labels-idx1-ubyte.gz', 'train-images-idx3-ubyte.gz',
      't10k-labels-idx1-ubyte.gz', 't10k-images-idx3-ubyte.gz'
  ]

  paths = []
  for fname in files:
    paths.append(os.path.join(input_path, fname))  # The location of the dataset
```

```
  with gzip.open(paths[0], 'rb') as lbpath:
    y_train = np.frombuffer(lbpath.read(), np.uint8, offset=8)

  with gzip.open(paths[1], 'rb') as imgpath:
    x_train = np.frombuffer(
        imgpath.read(), np.uint8, offset=16).reshape(len(y_train), 28, 28)

  with gzip.open(paths[2], 'rb') as lbpath:
    y_test = np.frombuffer(lbpath.read(), np.uint8, offset=8)

  with gzip.open(paths[3], 'rb') as imgpath:
    x_test = np.frombuffer(
        imgpath.read(), np.uint8, offset=16).reshape(len(y_test), 28, 28)

  return (x_train, y_train), (x_test, y_test)

# don't forget to change the path to your local path

(x_train,y_train),(x_test,y_test)=load_data_fromlocalpath('gz')
```

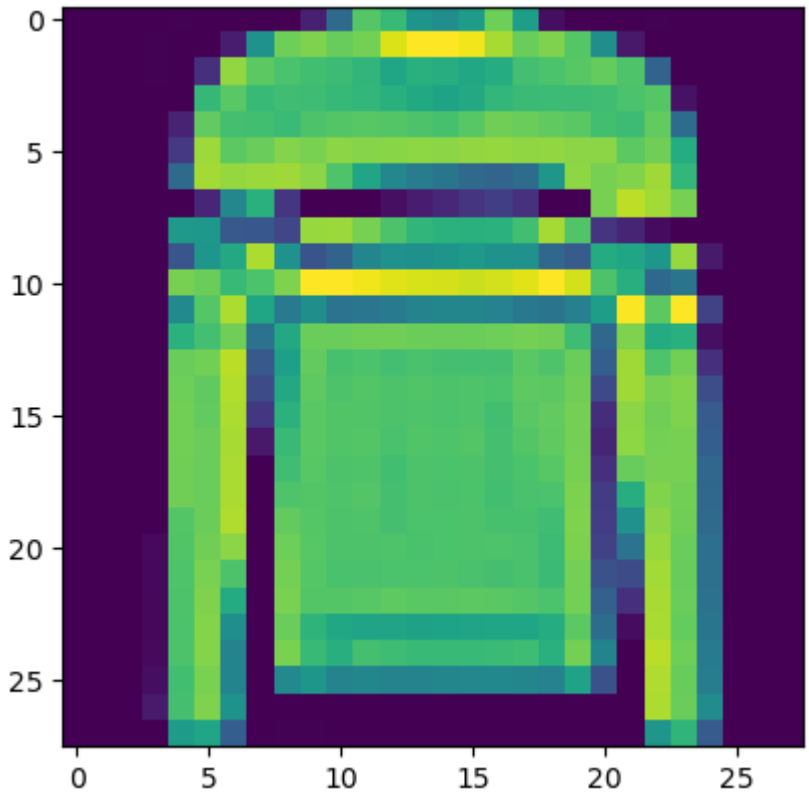
Visualize the data

In [48]:

```
# Print training set shape and explain the dataset
print("Training set shape:", x_train.shape)
print("Training set labels shape:", y_train.shape)
print("Test set shape:", x_test.shape)
print("Test set labels shape:", y_test.shape)

print("""
Dataset information:
Fashion MNIST is a Novel Image Dataset for Benchmarking Machine Learning Algorit
It contains 60,000 train and 10,000 test data with 10 categories.
Each gray-scale image is 28x28.""")

# Define the text labels
fashion_mnist_labels = ["T-shirt/top",  # index 0
                        "Trouser",      # index 1
                        "Pullover",     # index 2
                        "Dress",        # index 3
                        "Coat",         # index 4
                        "Sandal",       # index 5
                        "Shirt",        # index 6
                        "Sneaker",      # index 7
                        "Bag",          # index 8
                        "Ankle boot"]   # index 9

# Image index, you can pick any number between 0 and 59,999
img_index = 5
# y_train contains the lables, ranging from 0 to 9
label_index = y_train[img_index]
# Print the label, for example 2 Pullover
print ("y = " + str(label_index) + " " +(fashion_mnist_labels[label_index]))
# # Show one of the images from the training dataset
plt.imshow(x_train[img_index])
```

```
Training set shape: (60000, 28, 28)
Training set labels shape: (60000,)
Test set shape: (10000, 28, 28)
Test set labels shape: (10000,)

Dataset information:
Fashion MNIST is a Novel Image Dataset for Benchmarking Machine Learning Algorith
ms.
It contains 60,000 train and 10,000 test data with 10 categories.
Each gray-scale image is 28x28.
y = 2 Pullover
```

Out[48]: <matplotlib.image.AxesImage at 0x202230be630>



Data normalization

Normalize the data dimensions so that they are of approximately the same scale.

In [50]:
```
# Normalize the dataset (both the training and testing set)
x_train = x_train / 255.0
x_test = x_test / 255.0
```

Split the data into train/validation/test data sets

- Training data - used for training the model
- Validation data - used for tuning the hyperparameters and evaluate the models
- Test data - used to test the model after the model has gone through initial vetting by the validation set.

In [52]:
```
# Further break training data into train / validation sets (# put 5000 into vali
(x_train, x_valid) = x_train[5000:], x_train[:5000]
(y_train, y_valid) = y_train[5000:], y_train[:5000]

# Reshape input data from (28, 28) to (28, 28, 1)
w, h = 28, 28
x_train = x_train.reshape(x_train.shape[0], w, h, 1)
x_valid = x_valid.reshape(x_valid.shape[0], w, h, 1)
x_test = x_test.reshape(x_test.shape[0], w, h, 1)

# One-hot encode the labels
y_train = tf.keras.utils.to_categorical(y_train, 10)
y_valid = tf.keras.utils.to_categorical(y_valid, 10)
y_test = tf.keras.utils.to_categorical(y_test, 10)

# Print training set shape
print("x_train shape:", x_train.shape, "y_train shape:", y_train.shape)

# Print the number of training, validation, and test datasets
print(x_train.shape[0], 'train set')
print(x_valid.shape[0], 'validation set')
print(x_test.shape[0], 'test set')
```

```
x_train shape: (55000, 28, 28, 1) y_train shape: (55000, 10)
55000 train set
5000 validation set
10000 test set
```

Create the model architecture

There are two APIs for defining a model in Keras:

1. Sequential model API
2. Functional API

In this notebook we are using the Sequential model API. If you are interested in a tutorial using the Functional API, checkout Sara Robinson's blog Predicting the price of wine with the Keras Functional API and TensorFlow.

In defining the model we will be using some of these Keras APIs:

- Conv2D() link text - create a convolutional layer
- Pooling() link text - create a pooling layer
- Dropout() link text - apply drop out

In [54]:
```
model = tf.keras.Sequential()

# Must define the input shape in the first layer of the neural network
model.add(tf.keras.layers.Conv2D(filters=64, kernel_size=2, padding='same', acti
model.add(tf.keras.layers.MaxPooling2D(pool_size=2))
model.add(tf.keras.layers.Dropout(0.3))

model.add(tf.keras.layers.Conv2D(filters=32, kernel_size=2, padding='same', acti
model.add(tf.keras.layers.MaxPooling2D(pool_size=2))
model.add(tf.keras.layers.Dropout(0.3))
```

```
model.add(tf.keras.layers.Flatten())
model.add(tf.keras.layers.Dense(256, activation='relu'))
model.add(tf.keras.layers.Dropout(0.5))
model.add(tf.keras.layers.Dense(10, activation='softmax'))

# Take a look at the model summary
model.summary()
```

Model: "sequential_1"

Layer (type)	Output Shape	Param #
conv2d_2 (Conv2D)	(None, 28, 28, 64)	320
max_pooling2d_2 (MaxPooling2D)	(None, 14, 14, 64)	0
dropout_3 (Dropout)	(None, 14, 14, 64)	0
conv2d_3 (Conv2D)	(None, 14, 14, 32)	8,224
max_pooling2d_3 (MaxPooling2D)	(None, 7, 7, 32)	0
dropout_4 (Dropout)	(None, 7, 7, 32)	0
flatten_1 (Flatten)	(None, 1568)	0
dense_2 (Dense)	(None, 256)	401,664
dropout_5 (Dropout)	(None, 256)	0
dense_3 (Dense)	(None, 10)	2,570

Total params: 412,778 (1.57 MB)

Trainable params: 412,778 (1.57 MB)

Non-trainable params: 0 (0.00 B)

Compile the model

Configure the learning process with compile() API before training the model. It receives three arguments:

- An optimizer
- A loss function
- A list of metrics

In [56]:
```
model.compile(loss='categorical_crossentropy',
              optimizer='adam',
              metrics=['accuracy'])
```

Train the model

Now let's train the model with fit() API.

We use the ModelCheckpoint API to save the model after every epoch. Set "save_best_only = True" to save only when the validation accuracy improves.

In [58]:
```
from keras.callbacks import ModelCheckpoint

checkpointer = ModelCheckpoint(filepath='model.weights.best.h5', verbose = 1, sa
model.fit(x_train,
         y_train,
         batch_size=64,
         epochs=10,
         validation_data=(x_valid, y_valid),
         callbacks=[checkpointer])
```

```
Epoch 1/10
858/860 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - accuracy: 0.6796 - loss: 0.8560
Epoch 1: val_loss improved from inf to 0.39075, saving model to model.weights.bes
t.h5
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `ker
as.saving.save_model(model)`. This file format is considered legacy. We recommend
using instead the native Keras format, e.g. `model.save('my_model.keras')` or `ke
ras.saving.save_model(model, 'my_model.keras')`.
860/860 ━━━━━━━━━━━━━━━━━━━━ 11s 11ms/step - accuracy: 0.6799 - loss: 0.8552 - va
l_accuracy: 0.8648 - val_loss: 0.3907
Epoch 2/10
859/860 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - accuracy: 0.8384 - loss: 0.4426
Epoch 2: val_loss improved from 0.39075 to 0.32526, saving model to model.weight
s.best.h5
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `ker
as.saving.save_model(model)`. This file format is considered legacy. We recommend
using instead the native Keras format, e.g. `model.save('my_model.keras')` or `ke
ras.saving.save_model(model, 'my_model.keras')`.
860/860 ━━━━━━━━━━━━━━━━━━━━ 8s 10ms/step - accuracy: 0.8384 - loss: 0.4425 - val
_accuracy: 0.8902 - val_loss: 0.3253
Epoch 3/10
858/860 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - accuracy: 0.8614 - loss: 0.3780
Epoch 3: val_loss improved from 0.32526 to 0.29238, saving model to model.weight
s.best.h5
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `ker
as.saving.save_model(model)`. This file format is considered legacy. We recommend
using instead the native Keras format, e.g. `model.save('my_model.keras')` or `ke
ras.saving.save_model(model, 'my_model.keras')`.
860/860 ━━━━━━━━━━━━━━━━━━━━ 8s 10ms/step - accuracy: 0.8614 - loss: 0.3780 - val
_accuracy: 0.9004 - val_loss: 0.2924
Epoch 4/10
857/860 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - accuracy: 0.8696 - loss: 0.3558
Epoch 4: val_loss improved from 0.29238 to 0.28285, saving model to model.weight
s.best.h5
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `ker
as.saving.save_model(model)`. This file format is considered legacy. We recommend
using instead the native Keras format, e.g. `model.save('my_model.keras')` or `ke
ras.saving.save_model(model, 'my_model.keras')`.
860/860 ━━━━━━━━━━━━━━━━━━━━ 9s 10ms/step - accuracy: 0.8696 - loss: 0.3558 - val
_accuracy: 0.8986 - val_loss: 0.2828
Epoch 5/10
857/860 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - accuracy: 0.8806 - loss: 0.3289
Epoch 5: val_loss improved from 0.28285 to 0.26100, saving model to model.weight
s.best.h5
```

```
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `ker
as.saving.save_model(model)`. This file format is considered legacy. We recommend
using instead the native Keras format, e.g. `model.save('my_model.keras')` or `ke
ras.saving.save_model(model, 'my_model.keras')`.
860/860 ━━━━━━━━━━━━━━━━━━━━ 8s 10ms/step - accuracy: 0.8806 - loss: 0.3289 - val
_accuracy: 0.9048 - val_loss: 0.2610
Epoch 6/10
855/860 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - accuracy: 0.8857 - loss: 0.3117
Epoch 6: val_loss improved from 0.26100 to 0.25298, saving model to model.weight
s.best.h5
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `ker
as.saving.save_model(model)`. This file format is considered legacy. We recommend
using instead the native Keras format, e.g. `model.save('my_model.keras')` or `ke
ras.saving.save_model(model, 'my_model.keras')`.
860/860 ━━━━━━━━━━━━━━━━━━━━ 9s 11ms/step - accuracy: 0.8857 - loss: 0.3117 - val
_accuracy: 0.9070 - val_loss: 0.2530
Epoch 7/10
857/860 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - accuracy: 0.8910 - loss: 0.3026
Epoch 7: val_loss improved from 0.25298 to 0.24960, saving model to model.weight
s.best.h5
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `ker
as.saving.save_model(model)`. This file format is considered legacy. We recommend
using instead the native Keras format, e.g. `model.save('my_model.keras')` or `ke
ras.saving.save_model(model, 'my_model.keras')`.
860/860 ━━━━━━━━━━━━━━━━━━━━ 10s 12ms/step - accuracy: 0.8910 - loss: 0.3026 - va
l_accuracy: 0.9066 - val_loss: 0.2496
Epoch 8/10
856/860 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - accuracy: 0.8939 - loss: 0.2901
Epoch 8: val_loss improved from 0.24960 to 0.23709, saving model to model.weight
s.best.h5
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `ker
as.saving.save_model(model)`. This file format is considered legacy. We recommend
using instead the native Keras format, e.g. `model.save('my_model.keras')` or `ke
ras.saving.save_model(model, 'my_model.keras')`.
860/860 ━━━━━━━━━━━━━━━━━━━━ 9s 11ms/step - accuracy: 0.8939 - loss: 0.2901 - val
_accuracy: 0.9094 - val_loss: 0.2371
Epoch 9/10
858/860 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - accuracy: 0.8933 - loss: 0.2866
Epoch 9: val_loss improved from 0.23709 to 0.23003, saving model to model.weight
s.best.h5
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `ker
as.saving.save_model(model)`. This file format is considered legacy. We recommend
using instead the native Keras format, e.g. `model.save('my_model.keras')` or `ke
ras.saving.save_model(model, 'my_model.keras')`.
860/860 ━━━━━━━━━━━━━━━━━━━━ 9s 10ms/step - accuracy: 0.8933 - loss: 0.2866 - val
_accuracy: 0.9144 - val_loss: 0.2300
Epoch 10/10
859/860 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - accuracy: 0.9011 - loss: 0.2713
Epoch 10: val_loss improved from 0.23003 to 0.22667, saving model to model.weight
s.best.h5
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `ker
as.saving.save_model(model)`. This file format is considered legacy. We recommend
using instead the native Keras format, e.g. `model.save('my_model.keras')` or `ke
ras.saving.save_model(model, 'my_model.keras')`.
860/860 ━━━━━━━━━━━━━━━━━━━━ 9s 10ms/step - accuracy: 0.9011 - loss: 0.2713 - val
_accuracy: 0.9128 - val_loss: 0.2267
```

Out[58]:

```
<keras.src.callbacks.history.History at 0x2022303d7c0>
```

Load Model with the best validation accuracy

In [60]:
```
# Load the weights with the best validation accuracy
model.load_weights('model.weights.best.h5')
```

Test Accuracy

In [62]:
```
# Evaluate the model on test set
score = model.evaluate(x_test, y_test, verbose=0)

# Print test accuracy
print('Test accuracy:', score[1])
```

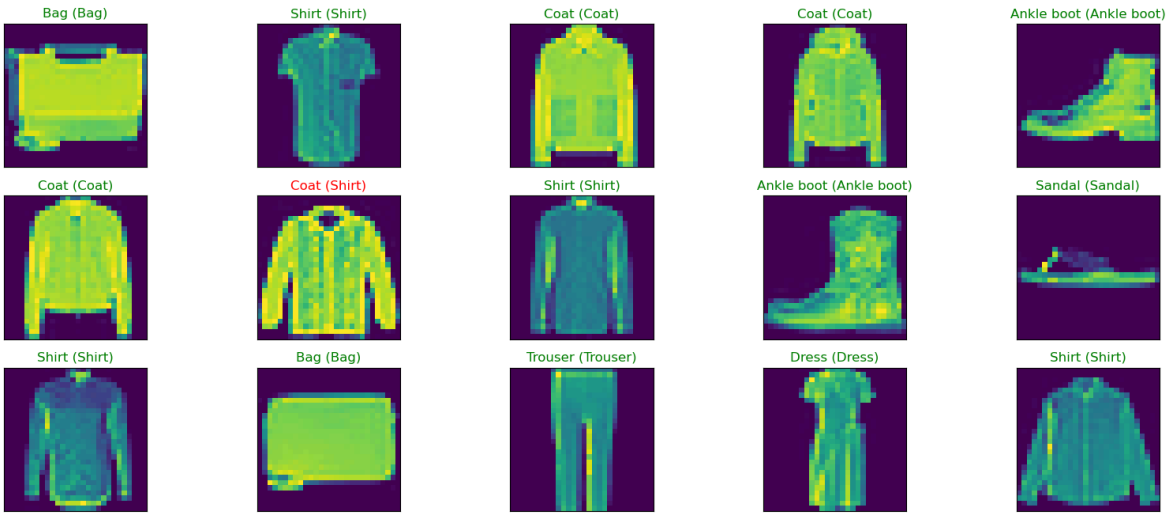
```
Test accuracy: 0.9101999998092651
```

Visualize prediction

Now let's visualize the prediction using the model you just trained. First we get the predictions with the model from the test data. Then we print out 15 images from the test data set, and set the titles with the prediction (and the groud truth label). If the prediction matches the true label, the title will be green; otherwise it's displayed in red.

In [64]:
```
y_hat = model.predict(x_test)

# Plot a random sample of 10 test images, their predicted labels and ground trut
figure = plt.figure(figsize=(20, 8))
for i, index in enumerate(np.random.choice(x_test.shape[0], size=15, replace=Fal
    ax = figure.add_subplot(3, 5, i + 1, xticks=[], yticks=[])
    # Display each image
    ax.imshow(np.squeeze(x_test[index]))
    predict_index = np.argmax(y_hat[index])
    true_index = np.argmax(y_test[index])
    # Set the title for each image
    ax.set_title("{} ({})".format(fashion_mnist_labels[predict_index],
                                  fashion_mnist_labels[true_index]),
                                  color=("green" if predict_index == true_index
```

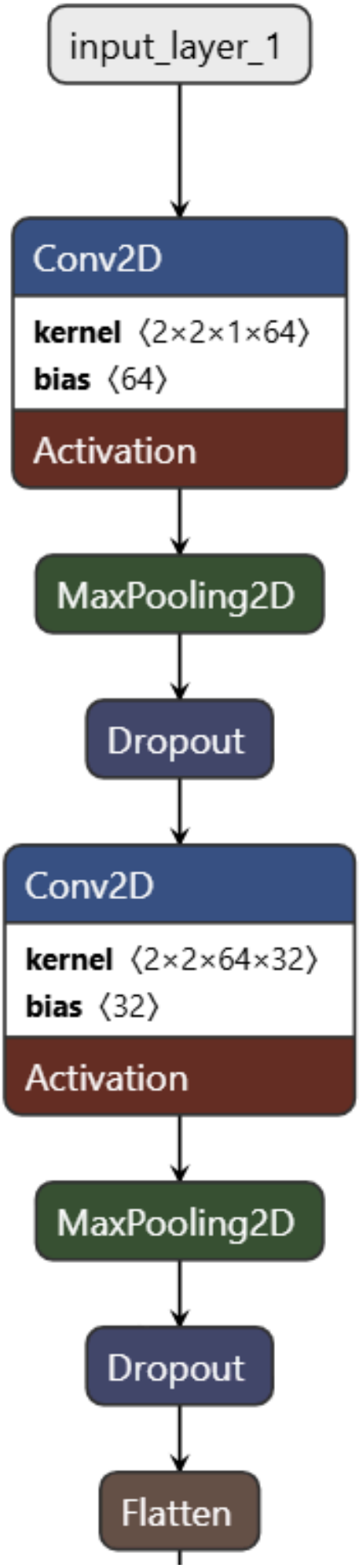
```
313/313 ━━━━━━━━━━━━━━━━━━━━ 1s 3ms/step
```

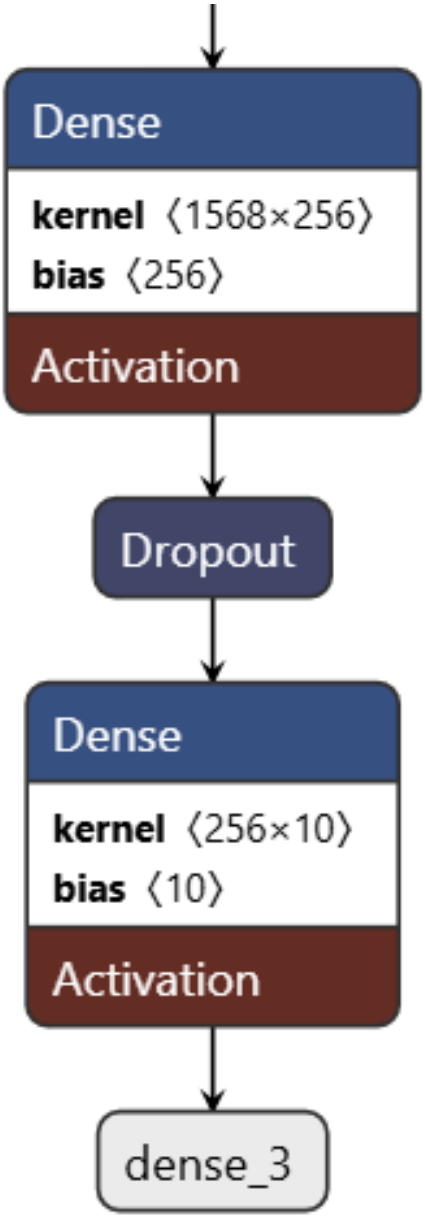


In [83]:

```
# visualize your model with netron https://netron.app/
# put the screenshot here
from IPython.display import Image

Image("snapshot.png", width=500)
```

Out[83]:


input_layer_1
Conv2D
kernel ⟨2×2×1×64⟩
bias ⟨64⟩
Activation
MaxPooling2D
Dropout
Conv2D
kernel ⟨2×2×64×32⟩
bias ⟨32⟩
Activation
MaxPooling2D
Dropout
Flatten



Congragulations!

You have successfully trained a CNN to classify fashion-MNIST with near 90% accuracy.

Note: If you cannot convert this file into pdf, the final solution is:

1. take screenshots page by page, paste them into a word file;
2. Convert the doc into pdf.

Submit your pdf file